Robust Parameter-Efficient LLM Adaptation on Analog Hardware

Jindan Li
Cornell University
j14767@cornell.edu

Zhaoxian Wu
Cornell University
zw868@cornell.edu

Tianyi Chen
Cornell University
tianyi.chen@cornell.edu

Abstract

Analog in-memory computing is a promising platform for on-device execution of large language models because it performs matrix–vector multiplications (MVMs) in memory and in parallel, reducing data movement. However, limited digital-to-analog converter precision, input noise, and finite conductance states can degrade model accuracy, while full-model retraining to address these effects can be costly. We develop an optimizer-agnostic, parameter-efficient adaptation method based on Low-Rank Adaptation (LoRA), keeping the pretrained weights stored on analog arrays fixed while training the LoRA weights to adapt to downstream tasks and hardware non-idealities. Reliable adaptation requires handling errors in both forward and backward MVMs and physical weight updates. We use input reshaping to reduce input-induced MVM errors and update accumulation to retain small updates before programming them to finite-state analog devices. Across Llama-3.2-1B-Instruct and Llama-3-8B with both Muon and AdamW, input reshaping improves analog LoRA fine-tuning under noisy MVM computation. Update accumulation separately preserves sub-threshold updates and substantially improves adaptation under finite-resolution programming, including configurations with as few as 20 conductance states. Additional experiments show consistent held-out negative log-likelihood improvements across noisy analog settings.

1 Introduction

Large language models (LLMs) rely heavily on matrix–vector multiplications (MVMs) in their linear layers [1, 2]. Analog in-memory computing (AIMC) [3, 4] is an attractive hardware platform for these operations because weights can be stored directly in memory and MVMs can be performed in place and in parallel [5–7]. This reduces data movement and can improve energy efficiency and latency [8, 7, 9], making AIMC a promising candidate for on-device intelligence.

However, deploying pretrained LLMs on analog hardware can degrade model accuracy because limited digital-to-analog converter (DAC) precision, input noise, and finite conductance states introduce errors into the model's computation. Hardware-aware retraining can mitigate this degradation by accounting for analog non-idealities during training [9–11]. However, retraining the full model requires gradients and optimizer states for all trainable weights, as well as computation to update those weights. Repeating this process for downstream tasks can therefore incur substantial training-memory and computation costs. These costs motivate parameter-efficient fine-tuning (PEFT) for on-device analog adaptation [12]. We consider Low-Rank Adaptation (LoRA) [13, 14] to adapt the model to downstream tasks and compensate for hardware-induced degradation while keeping the pretrained weights fixed. In our setting, the deployed pretrained weights and the corresponding LoRA weights are executed on analog arrays, with digital support for input reshaping and update accumulation.

Reducing the number of trainable weights does not remove the errors introduced by analog computation and programming. Limited DAC precision and input noise perturb the forward and backward

NeurIPS 2026 Workshop on On-Device Intelligence: Foundation Models under Real-World Constraints.

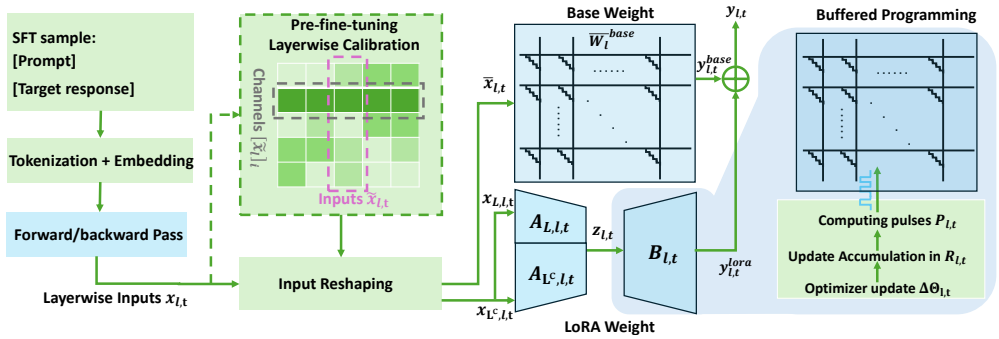


Figure 1: **Overview of analog LoRA fine-tuning.** Pre-fine-tuning calibration provides channel statistics for input reshaping. For the frozen base path, function-preserving base-path rescaling reduces the input range. For the trainable LoRA path, outlier-channel splitting routes persistent outlier channels through the protected analog path and keeps the remaining channels on the noisy analog path. During weight programming, digital mini-batch aggregation combines sample gradients, and the update accumulator stores sub-threshold updates until they can be converted into programming pulses. Blue and green denote analog and digital computation, respectively.

MVMs used to compute LoRA gradients. Finite conductance states and non-ideal programming also limit how accurately the resulting updates can be written to the LoRA weights [5, 15, 10]. Our input reshaping treats the fixed pretrained weights and trainable LoRA weights differently. For the pretrained weights, function-preserving base-path rescaling jointly rescales the weights and their inputs to reduce the input range without changing the ideal computation [16]. For the LoRA weights, we route persistent input-outlier channels through a protected perfect-I/O analog path, so that their large values no longer set the quantization scale for the remaining analog inputs. For weight programming, we use mixed-precision update stabilization: digital mini-batch aggregation combines gradients before the optimizer step, and update accumulation retains updates smaller than a conductance step until they can be programmed [7]. Figure 1 summarizes the method.

Our contributions are threefold. First, we develop a lightweight adaptation method for LLMs on analog hardware, motivated by the needs of on-device intelligence. The method keeps pretrained weights fixed and trains only a small set of LoRA weights to address hardware-induced degradation. Second, we address noisy MVM computation through input reshaping and limited weight-update resolution through mixed-precision update stabilization using digital mini-batch aggregation and update accumulation. Third, we evaluate the optimizer-agnostic design in simulation on Llama-3.2-1B-Instruct and Llama-3-8B using Muon [17, 18] and AdamW [19]. Input reshaping remains effective with both optimizers under noisy MVM computation, while its gains largely disappear when MVM input perturbations are removed. Update accumulation separately improves adaptation under finite-resolution weight programming. Additional model and task results show consistent held-out negative log-likelihood improvements under noisy analog settings.

2 Related Work

Parameter-efficient fine-tuning. Parameter-efficient fine-tuning methods include Adapter Tuning [20], Prefix Tuning [21], Prompt Tuning [22], LoRA [13], QLoRA [23], AdaLoRA [24], and DoRA [25]. Recent work further revisits LoRA architectures, optimization, and deployment considerations from a signal-processing perspective [26]. For analog hardware adaptation, prior work keeps the pretrained weights fixed on AIMC tiles while performing the LoRA computation on separate digital processors [14]. Its deployment therefore requires a dedicated digital LoRA execution path whose latency must be balanced with the AIMC tiles. We instead place the main LoRA computation and weight updates on analog arrays. This removes the need to rely on a fully digital LoRA computation path, but makes the trainable LoRA weights themselves subject to noisy MVMs and non-ideal programming, which motivates the robustness techniques studied in this work.

Analog training and low-precision computation. Analog in-memory training is affected by noisy MVMs, finite conductance states [27], peripheral quantization, and non-ideal programming

responses [28–30]. Hardware-aware training and simulation frameworks model these effects during training [9–11], while mixed-precision computational-memory training uses high-precision digital accumulation to improve weight updates on low-precision analog devices [7]. Recent work further co-designs weight mapping and optimization to improve the scalability of analog training under hardware non-idealities [31]. Complementary work on low-precision transformers addresses large activation ranges and persistent outliers through activation–weight rescaling [16], outlier suppression [32], and activation-aware quantization [33]. Noise-optimized rescaling (NoRA) further applies activation–weight rescaling to analog inference to reduce input-side non-idealities [34].

3 Analog LoRA Fine-Tuning under Hardware Non-Idealities

This section presents our analog LoRA fine-tuning method under non-ideal MVM computation and physical weight programming. We first define the analog LoRA computation and the hardware-noise model used during training. We then introduce input reshaping to reduce DAC-side MVM perturbations, followed by digital mini-batch aggregation and update accumulation to stabilize finite-state programming of the trainable LoRA weights.

3.1 Analog LoRA and non-ideal MVMs

We start from a pretrained transformer and map part of the pretrained weights and the corresponding trainable LoRA weights to analog arrays. For a layer with input dimension n and output dimension m, the pretrained base weights $\mathbf{W}^{\text{base}} \in \mathbb{R}^{m\times n}$ are programmed once and kept fixed, while the LoRA matrices $\mathbf{A} \in \mathbb{R}^{r\times n}$ and $\mathbf{B} \in \mathbb{R}^{m\times r}$ are updated during fine-tuning. Both the pretrained and LoRA weights are used in non-ideal analog forward and backward MVMs. Because the LoRA weights are trainable, they are additionally subject to finite-state and non-ideal physical programming when updates are written to the arrays. For input $\mathbf{x} \in \mathbb{R}^n$, the ideal output $\mathbf{y} \in \mathbb{R}^m$ is

$$\mathbf{y} = \mathbf{W}^{\text{base}}\mathbf{x} + \frac{\alpha}{r}\mathbf{B}\mathbf{A}\mathbf{x}, \tag{1}$$

where r is the LoRA rank and α is the LoRA scaling factor.

Our input reshaping is motivated by the persistent activation outliers observed in pretrained transformers [16, 33]. We use absolute-maximum input scaling, which normalizes each MVM input by its maximum absolute value $\|\mathbf{x}\|_\infty$ so that the largest input magnitude determines the DAC range. As a result, a few persistent outlier channels can repeatedly enlarge the quantization step for all remaining channels. We therefore reshape the MVM inputs to reduce this dynamic range while preserving the intended computation. For a general weight $\mathbf{W}$, we model a non-ideal analog MVM as $f_{\text{out}}(\mathbf{W} f_{\text{DAC}}(\mathbf{x}))$, where $f_{\text{DAC}}(\cdot)$ captures input-side conversion and noise, and $f_{\text{out}}(\cdot)$ captures output-side readout effects. The effective MVM input after DAC conversion is modeled as

$$f_{\text{DAC}}(\mathbf{x}) = \|\mathbf{x}\|_\infty \left(\frac{\mathbf{x}}{\|\mathbf{x}\|_\infty} \odot (\mathbf{1} + \mathbf{m}_\mathbf{x}) + \mathbf{q}_\mathbf{x} \right) = \mathbf{x} \odot (\mathbf{1} + \mathbf{m}_\mathbf{x}) + \|\mathbf{x}\|_\infty \mathbf{q}_\mathbf{x} := \mathbf{x} + \boldsymbol{\varepsilon}_\mathbf{x}. \tag{2}$$

Here $\odot$ denotes element-wise multiplication, $\mathbf{1}$ is the all-ones vector, and $\boldsymbol{\varepsilon}_\mathbf{x}$ is the total input perturbation. The first expression makes the DAC-domain perturbation explicit: the input is first normalized by $\|\mathbf{x}\|_\infty$ before entering the DAC, element-wise multiplicative noise $\mathbf{m}_\mathbf{x}$ and quantization error $\mathbf{q}_\mathbf{x}$ are applied in the normalized domain, and the result is then scaled back to the original input range. For each input dimension $j \in \{1, \dots, n\}$, the multiplicative noise is modeled as $[\mathbf{m}_\mathbf{x}]_j \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_x^2)$, where σ_x controls the input-noise level. The DAC quantization error is modeled as $[\mathbf{q}_\mathbf{x}]_j \overset{\text{i.i.d.}}{\sim} \text{Unif}\left[-\frac{1}{2^{b_{\text{DAC}}}-2}, \frac{1}{2^{b_{\text{DAC}}}-2}\right]$, where b_{DAC} is the DAC precision in bits. The same input-noise model is used for analog backward MVMs, with the backpropagated error replacing $\mathbf{x}$. The programming non-idealities associated with updating the trainable LoRA weights are described separately in Sec. 3.3.

3.2 Input reshaping

We use a one-time pre-fine-tuning calibration to identify persistent high-magnitude input channels, and use these statistics to reshape the inputs seen by the low-precision analog paths. Input reshaping is applied differently to the frozen base weights and the trainable LoRA weights.

For the frozen base path, we use function-preserving base-path rescaling. Before fine-tuning, we estimate input-channel magnitudes from calibration batches separately for each layer to which reshaping is applied. Let $|[\widetilde{\mathbf{x}}]_i|$ denote the resulting magnitude statistic for input channel $i = 1, \ldots, n$. We construct a diagonal rescaling matrix $\mathbf{D} \propto \mathrm{diag}\big(|[\widetilde{\mathbf{x}}]_1|^\lambda, \ldots, |[\widetilde{\mathbf{x}}]_n|^\lambda\big)$, where $\lambda \in (0, 1]$ controls the rescaling strength. The calibration statistics and $\mathbf{D}$ remain fixed throughout fine-tuning. The input and the corresponding frozen weights are rescaled together as

$$\mathbf{W}^{\mathrm{base}}\mathbf{x} = (\mathbf{W}^{\mathrm{base}}\mathbf{D})(\mathbf{D}^{-1}\mathbf{x}), \qquad \overline{\mathbf{W}}^{\mathrm{base}} := \mathbf{W}^{\mathrm{base}}\mathbf{D}, \qquad \overline{\mathbf{x}} := \mathbf{D}^{-1}\mathbf{x}. \tag{3}$$

Thus, the ideal base-path computation is unchanged, while persistent high-magnitude input channels are reduced in $\overline{\mathbf{x}}$. Because the base weights are frozen, $\overline{\mathbf{W}}^{\mathrm{base}}$ is programmed to the analog array once before fine-tuning. The benefit follows directly from the DAC-noise model in Eq. (2). After rescaling, the analog input perturbation is $\boldsymbol{\varepsilon}_{\overline{\mathbf{x}}} = \overline{\mathbf{x}} \odot \mathbf{m}_{\overline{\mathbf{x}}} + \|\overline{\mathbf{x}}\|_\infty \mathbf{q}_{\overline{\mathbf{x}}}$. The rescaling is designed to reduce the input quantization-noise scale $\|\overline{\mathbf{x}}\|_\infty$. The output-side effect also depends on the rescaled weights; Figure 3 reports the resulting noise ratios.

For the trainable LoRA path, we use outlier-channel splitting. Using the same calibration statistics, we select a persistent outlier-channel set $\mathcal{L}$. The channels in $\mathcal{L}$ are computed on the protected path, while the remaining channels $\mathcal{L}^c = \{1, \ldots, n\} \setminus \mathcal{L}$ are kept on the low-precision analog path. The vectors $\mathbf{x}_\mathcal{L}$ and $\mathbf{x}_{\mathcal{L}^c}$ contain the corresponding input channels, and $\mathbf{A}_\mathcal{L}$ and $\mathbf{A}_{\mathcal{L}^c}$ contain the corresponding columns of $\mathbf{A}$. The A-side LoRA output is

$$\mathbf{z} = \underbrace{\mathbf{A}_\mathcal{L}\mathbf{x}_\mathcal{L}}_{\text{protected analog path}} + \underbrace{\mathbf{A}_{\mathcal{L}^c}\left(\mathbf{x}_{\mathcal{L}^c} \odot (\mathbf{1} + \mathbf{m}_{\mathcal{L}^c}) + \|\mathbf{x}_{\mathcal{L}^c}\|_\infty \mathbf{q}_{\mathcal{L}^c}\right)}_{\text{low-precision analog path}}. \tag{4}$$

Removing the persistent outlier channels from the analog path replaces the DAC range $\|\mathbf{x}\|_\infty$ by $\|\mathbf{x}_{\mathcal{L}^c}\|_\infty$. The analog-input perturbation is therefore $\boldsymbol{\varepsilon}_{\mathcal{L}^c} = \mathbf{x}_{\mathcal{L}^c} \odot \mathbf{m}_{\mathcal{L}^c} + \|\mathbf{x}_{\mathcal{L}^c}\|_\infty \mathbf{q}_{\mathcal{L}^c}$. Because the calibrated persistent outlier channels are excluded from the analog path, $\|\mathbf{x}_{\mathcal{L}^c}\|_\infty \le \|\mathbf{x}\|_\infty$, with a strict reduction whenever a dominant input channel belongs to $\mathcal{L}$. This reduces the DAC-quantization term on the analog LoRA path in those cases. The selected channels are computed on the protected perfect-I/O analog path and are therefore not subject to the modeled input perturbation in Eq. (2).

3.3 Update-path stabilization for analog LoRA

The trainable LoRA weights are also affected by non-ideal physical programming. Finite conductance resolution limits how small an update can be programmed to the analog LoRA weights. We use digital mini-batch aggregation and update accumulation [7]. Gradients are aggregated digitally before the optimizer computes the weight update, and the same update accumulation procedure is then applied to the resulting update regardless of the optimizer choice. Updates smaller than the conductance step are retained until they can be programmed.

Let t denote an optimizer step and $\boldsymbol{\Theta}_t \in \{\mathbf{A}_t, \mathbf{B}_t\}$ denote either trainable LoRA weight matrix. Using a uniform update resolution over $[-\tau, \tau]$, where τ is the weight-range bound, we set $\Delta w_{\min} = 2\tau/N$. Let $\Delta\boldsymbol{\Theta}_t$ be the signed weight update produced by the optimizer, instantiated as either Muon or AdamW in our experiments, after digital mini-batch aggregation [31]. A digital accumulator $\mathbf{R}_t$ then stores the optimizer update before it is converted into an integer number of programming pulses:

$$\mathbf{R}_{t+\frac{1}{2}} = \mathbf{R}_t + \Delta\boldsymbol{\Theta}_t, \quad \mathbf{P}_t = \mathrm{trunc}\Big(\mathbf{R}_{t+\frac{1}{2}}/\Delta w_{\min}\Big), \quad \mathbf{R}_{t+1} = \mathbf{R}_{t+\frac{1}{2}} - \Delta w_{\min}\mathbf{P}_t. \tag{5}$$

Here $\mathrm{trunc}(\cdot)$ is applied element-wise toward zero. $\mathbf{R}_{t+\frac{1}{2}}$ denotes the accumulator after adding the current update, and $\mathbf{P}_t$ is the signed integer pulse-count matrix.

4 Experiments

We evaluate robust analog LoRA fine-tuning using IBM Analog Hardware Acceleration Kit (AIHWKit) [11]. We consider three main testbeds spanning question answering (QA) and instruction tuning: the Llama-3-8B, Llama-3.2-1B-Instruct, and Qwen2.5-7B-Instruct. For the two QA testbeds, we use an equal mixture of NQ-Open, TriviaQA, and WebQuestions [35–38]. For Llama-3-8B, the Q/K/V/O projections and corresponding LoRA weights in the first six decoder blocks are mapped to analog arrays; for Llama-3.2-1B-Instruct, the corresponding projections in all 16 decoder blocks

Table 1: **Primary QA results.** Each entry reports F1/EM (%). Reshaping denotes base-path rescaling + outlier-channel splitting. UA denotes update accumulation. I/O b/s denotes a b-bit input DAC with input-noise standard deviation s.

Model	I/O	Optimizer	No reshaping + UA	Reshaping + UA	Reshaping w/o UA
Llama-3-8B	Perfect	Muon	46.84 / 37.89	47.84 / 39.26	35.51 / 26.17
Llama-3-8B	Perfect	AdamW	48.50 / 40.63	48.36 / 40.43	35.51 / 26.17
Llama-3-8B	7/0.02	Muon	24.49 / 18.36	**43.55 / 35.35**	28.73 / 19.92
Llama-3-8B	7/0.02	AdamW	31.51 / 24.22	**43.86 / 35.94**	30.32 / 20.90
Llama-3.2-1B	Perfect	Muon	30.88 / 23.24	30.10 / 24.22	10.61 / 0.00
Llama-3.2-1B	Perfect	AdamW	28.19 / 21.09	29.34 / 21.88	10.61 / 0.00
Llama-3.2-1B	7/0.02	Muon	19.21 / 12.89	**25.24 / 17.97**	8.92 / 0.00
Llama-3.2-1B	7/0.02	AdamW	16.48 / 11.13	**23.70 / 16.80**	9.02 / 0.00

are mapped to analog arrays. Training uses 2,000 optimizer steps, sequence length 256, LoRA rank $r = 8$, and scaling factor $\alpha = 16$. Input reshaping is applied to Q/K/V using one-time pre-fine-tuning calibration over 64 batches, with $\lambda = 0.8$ for base-path rescaling and the eight most persistent high-magnitude channels selected for outlier-channel splitting. The primary analog configuration uses $N = 100$, 7-bit input DACs, and input-noise standard deviation 0.02 in both forward and backward MVMs. Perfect-I/O controls remove these modeled MVM input perturbations. We report F1 and Exact Match (EM) using greedy generation on a fixed 512-example QA set, together with response-token negative log-likelihood (NLL) on 100 held-out examples.

For instruction tuning, we fine-tune `Qwen2.5-7B-Instruct` on `OpenThoughts-114k` using the Qwen chat template [39]. The Q/K/V projections and corresponding LoRA weights in the last six decoder blocks are mapped to analog arrays. Input reshaping uses fixed pre-fine-tuning calibration with $\lambda = 0.8$ and eight persistent high-magnitude input channels, with the selected channels routed through the same protected perfect-I/O analog path used in the primary experiments. We evaluate held-out assistant-response-token NLL across multiple conductance-resolution and DAC/noise settings. Additional implementation details and supplementary testbeds are provided in Appendix A.

4.1 Question-Answering Results

The QA experiments address two complementary questions. The first two result columns in Table 1 evaluate input reshaping. Under noisy MVM computation, reshaping improves F1 and EM for both models and both optimizers, while its effect is small under perfect I/O, where the targeted input-side perturbations are removed. The Llama-3.2-1B-Instruct results further show that the benefit persists when analog mapping is extended to all 16 decoder blocks.

The last two result columns in Table 1 isolate update accumulation while keeping reshaping enabled. Discarding the programming residual substantially degrades F1 and EM for both models and optimizers, demonstrating the importance of retaining sub-threshold updates under finite-resolution programming. Validation NLL in Table 4 shows the same qualitative trends for both input reshaping and update accumulation. The $N = 20$ stress test in Table 5 further shows that both benefits persist under coarser programming resolution.

4.2 Instruction-Tuning Results

Table 2 evaluates input reshaping on Qwen2.5-7B-Instruct across multiple noise settings and two conductance resolutions. Reshaping lowers mean held-out NLL in all eight noisy within-optimizer comparisons for both Muon and AdamW. Together with the Llama QA results, these results show that input reshaping remains effective across different models, tasks, optimizers, and analog settings.

4.3 Training-Time versus Evaluation-Only Reshaping

Motivated by analog-inference methods such as NORA [34], Table 3 examines whether input reshaping is useful only as an inference-time correction, or whether applying it during fine-tuning

Table 2: **Qwen/OpenThoughts held-out NLL.** Mean ± standard deviation over three seeds; lower is better. UA is enabled throughout.

N	I/O setting	Muon [NLL↓]		AdamW [NLL↓]	
		No reshaping	Reshaping	No reshaping	Reshaping
–	Digital	0.6823±0.0003	–	0.7235±0.0001	–
100	Perfect	0.7121±0.0006	–	0.7548±0.0013	–
20	Perfect	0.7896±0.0025	–	0.8727±0.0000	–
100	4/0.02	0.7356±0.0009	**0.7209**±0.0001	0.7852±0.0005	**0.7649**±0.0007
100	6/0.01	0.7154±0.0004	**0.7104**±0.0002	0.7569±0.0007	**0.7521**±0.0005
20	4/0.02	0.8158±0.0019	**0.7952**±0.0010	0.9128±0.0001	**0.8908**±0.0004
20	5/0.03	0.8051±0.0029	**0.7911**±0.0018	0.8936±0.0005	**0.8812**±0.0002

Table 3: **Training-time versus evaluation-only reshaping.** F1 (%) is reported as mean ± sample standard deviation over three repeated noisy evaluations. Without adaptation, reshaping only uses base-path rescaling (BR). With adaptation, reshaping uses both BR and outlier-channel splitting.

Setting	Optimizer	No reshaping	Eval-only reshaping	Train+eval reshaping
No adaptation	–	7.12 ± 0.07	9.14 ± 0.09	–
Adaptation	AdamW	18.20 ± 1.06	22.02 ± 0.26	$\mathbf{23.57 \pm 0.28}$
	Muon	19.52 ± 0.73	21.82 ± 0.67	$\mathbf{24.41 \pm 0.58}$

provides an additional benefit. We use Llama-3.2-1B-Instruct under the primary noisy-I/O setting ($N = 100$, 7-bit input DAC, and input noise 0.02), with three repeated noisy evaluations on the same 512 QA examples. Without adaptation, applying function-preserving base-path rescaling only at inference already improves the F1 score of frozen model. We next evaluate the same question after LoRA adaptation, where input reshaping combines function-preserving base-path rescaling and outlier-channel splitting. With analog LoRA adaptation, reshaping already improves F1 when applied only at evaluation, from 18.20 to 22.02 for AdamW and from 19.52 to 21.82 for Muon. Applying reshaping during both fine-tuning and evaluation further increases F1 to 23.57 and 24.41, respectively. Together, these results show that input reshaping benefits both inference and analog LoRA fine-tuning, while the large improvement over the frozen-model result (9.14 F1 score) demonstrates effective task adaptation under noisy analog computation.

The supplementary experiments further examine generalization and the individual components of the method. On Llama-3-8B/OpenOrca, Table 6 reports lower mean response-token NLL with reshaping across all noisy settings tested. The recovered linear-attention Llama-3-8B experiments in Tables 7 and 8 provide an additional architecture variant, with F1 and EM improving in most noisy comparisons across multiple conductance resolutions. We also provide diagnostics to better understand the reshaping mechanism. Table 9 evaluates the individual contributions of base-path rescaling and outlier-channel splitting. Figure 2 examines whether the channels selected during one-time calibration remain persistent during fine-tuning. Finally, Fig. 3 reports analytical noise ratios associated with the two reshaping components. Together, these supplementary results complement the main-task results with additional task and architecture coverage, component ablations, and mechanism-level diagnostics.

5 Conclusion and Limitations

We present a parameter-efficient approach for adapting pretrained LLMs to analog hardware without full-model hardware-aware retraining. Input reshaping mitigates input-side MVM errors, while update accumulation preserves sub-threshold updates under finite-resolution programming. Across the primary QA and additional evaluations, these techniques improve analog LoRA fine-tuning under the simulated non-idealities considered. Our study is limited to AIHWKit simulation and selected models and device settings; it does not measure physical-device behavior or end-to-end latency, energy, programming cost, or communication overhead. Some supplementary stress-test and component-ablation results are point estimates. Physical validation, broader device coverage, and system-level evaluation remain future work.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[2] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," *Conference on Machine Learning and Systems*, 2023.

[3] D. Ielmini and H.-S. P. Wong, "In-memory computing with resistive switching devices," *Nature electronics*, vol. 1, no. 6, pp. 333–343, 2018.

[4] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.

[5] T. Gokmen and Y. Vlasov, "Acceleration of deep neural network training with resistive cross-point devices: Design considerations," *Frontiers in neuroscience*, vol. 10, p. 333, 2016.

[6] S. Ambrogio, P. Narayanan, H. Tsai, R. M. Shelby, I. Boybat, C. Di Nolfo, S. Sidler, M. Giordano, M. Bodini, N. C. Farinha *et al.*, "Equivalent-accuracy accelerated neural-network training using analogue memory," *Nature*, vol. 558, no. 7708, pp. 60–67, 2018.

[7] S. Nandakumar, M. Le Gallo, C. Piveteau, V. Joshi, G. Mariani, I. Boybat, G. Karunaratne, R. Khaddam-Aljameh, U. Egger, A. Petropoulos *et al.*, "Mixed-precision deep learning based on computational memory," *Frontiers in neuroscience*, vol. 14, p. 406, 2020.

[8] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," *IEEE International Solid-State Circuits Conference*, pp. 10–14, 2014.

[9] M. J. Rasch, C. Mackin, M. Le Gallo, A. Chen, A. Fasoli, F. Odermatt, N. Li, S. Nandakumar, P. Narayanan, H. Tsai *et al.*, "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators," *Nature communications*, vol. 14, no. 1, p. 5282, 2023.

[10] M. Le Gallo, C. Lammie, J. Büchel, F. Carta, O. Fagbohungbe, C. Mackin, H. Tsai, V. Narayanan, A. Sebastian, K. El Maghraoui *et al.*, "Using the ibm analog in-memory hardware acceleration kit for neural network training and inference," *APL Machine Learning*, vol. 1, no. 4, 2023.

[11] J. Büchel, W. A. Simon, C. Lammie, G. Acampa, K. El Maghraoui, M. Le Gallo, and A. Sebastian, "Aihwkit-lightning: a scalable hw-aware training toolkit for analog in-memory computing," in *NeurIPS 2024 Workshop Machine Learning with new Compute Paradigms*, 2024.

[12] N. Ding, Y. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Hu, Y. Chen, C.-M. Chan, W. Chen *et al.*, "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature machine intelligence*, vol. 5, no. 3, pp. 220–235, 2023.

[13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *International Conference on Learning Representations*, 2022.

[14] C. Li, E. Ferro, C. Lammie, M. Le Gallo, I. Boybat, and B. Rajendran, "Efficient transformer adaptation for analog in-memory computing via low-rank adapters," *Neuromorphic Computing and Engineering*, vol. 6, no. 1, p. 014011, 2026.

[15] T. Gokmen and W. Haensch, "Algorithm for training neural networks on resistive device arrays," *Frontiers in neuroscience*, vol. 14, p. 103, 2020.

[16] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," *International conference on machine learning*, pp. 38 087–38 099, 2023.

[17] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/

[18] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan *et al.*, "Muon is scalable for llm training," *arXiv preprint arXiv:2502.16982*, 2025.

[19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations*, 2019.

[20] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," *International Conference on Machine Learning*, pp. 2790–2799, 2019.

[21] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *Annual Meeting of the Association for Computational Linguistics and International Joint Conference on Natural Language Processing*, pp. 4582–4597, 2021.

[22] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," *Conference on Empirical Methods in Natural Language Processing*, pp. 3045–3059, 2021.

[23] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized LLMs," *Advances in neural information processing systems*, vol. 36, pp. 10 088–10 115, 2023.

[24] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," *International Conference on Learning Representations*, 2023.

[25] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, "Dora: Weight-decomposed low-rank adaptation," *International Conference on Machine Learning*, 2024.

[26] B. Li, Y. Zhang, and G. B. Giannakis, "Low-rank adaptation redux for large models," *arXiv preprint arXiv:2604.21905*, 2026.

[27] J. Li, Z. Wu, G. Liu, T. Gokmen, and T. Chen, "In-memory training on analog devices with limited conductance states via multi-tile residual learning," *International Conference on Artificial Intelligence and Statistics*, pp. 3664–3672, 2026.

[28] Z. Wu, T. Gokmen, M. J. Rasch, and T. Chen, "Towards exact gradient-based training on analog in-memory computing," *Advances in Neural Information Processing Systems*, vol. 37, pp. 37 264–37 304, 2024.

[29] Z. Wu, Q. Xiao, T. Gokmen, O. Fagbohungbe, and T. Chen, "Analog in-memory training on general non-ideal resistive elements: The impact of response functions," *Advances in Neural Information Processing Systems*, vol. 38, Main Conference, 2025.

[30] Q. Xiao, J. Li, Z. Wu, T. Gokmen, and T. Chen, "Dynamic symmetric point tracking: Tackling non-ideal reference in analog in-memory training," *International Conference on Machine Learning*, pp. 138 794–138 829, 2026.

[31] Z. Wu, T. Gokmen, O. Fagbohungbe, T. P. Xiao, and T. Chen, "Making analog training scale: Co-designing mapping, optimizer, and converters," 2026. [Online]. Available: https://arxiv.org/abs/2609.36584

[32] X. Wei, Y. Zhang, X. Zhang, R. Gong, S. Zhang, Q. Zhang, F. Yu, and X. Liu, "Outlier suppression: Pushing the limit of low-bit transformer language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 402–17 414, 2022.

[33] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.

[34] Y. Hou, H. Tsai, K. El Maghraoui, T. Gokmen, G. W. Burr, and L. Liu, "Nora: Noise-optimized rescaling of llms on analog compute-in-memory accelerators," *Design, Automation & Test in Europe Conference*, pp. 1–7, 2025.

[35] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee *et al.*, "Natural questions: a benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.

[36] K. Lee, M.-W. Chang, and K. Toutanova, "Latent retrieval for weakly supervised open domain question answering," *Annual Meeting of the Association for Computational Linguistics*, pp. 6086–6096, 2019.

[37] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," *Annual Meeting of the Association for Computational Linguistics*, pp. 1601–1611, 2017.

[38] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on freebase from question-answer pairs," *Conference on Empirical Methods in Natural Language Processing*, pp. 1533–1544, 2013.

[39] E. Guha, R. Marten, S. Keh, N. Raoof, G. Smyrnis, H. Bansal, M. Nezhurina, J. Mercat, T. Vu, Z. Sprague *et al.*, "Openthoughts: Data recipes for reasoning models," *International Conference on Learning Representations*, 2026.

[40] M. J. Rasch, D. Moreda, T. Gokmen, M. Le Gallo, F. Carta, C. Goldberg, K. El Maghraoui, A. Sebastian, and V. Narayanan, "A flexible and fast pytorch toolkit for simulating training and inference on analog crossbar arrays," *IEEE International Conference on Artificial Intelligence Circuits and Systems*, pp. 1–4, 2021.

[41] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[42] M. Zhang, S. Arora, R. Chalamala, B. F. Spector, A. Wu, K. Ramesh, A. Singhal, and C. Ré, "LoLCATs: On low-rank linearizing of large language models," *International Conference on Learning Representations*, 2025.

Supplementary Material

A Experimental Details

We use IBM AIHWKit [40, 10] to simulate analog LoRA fine-tuning. Pretrained weights remain frozen, while the LoRA weights in the mapped layers are trainable. DAC precision and input noise are applied to both forward and backward analog MVMs. Digital mini-batch aggregation is performed before the Muon or AdamW optimizer step, followed by the update-accumulation procedure described in Sec. 3.3. Throughout the paper, *No reshaping + UA* disables input reshaping while retaining update accumulation (UA), *Reshaping + UA* enables both components, and *Reshaping w/o UA* enables reshaping but discards the programming residual after each transfer. Perfect-I/O analog controls remove the modeled MVM input perturbations while retaining the analog weight representation and programming procedure. I/O b/s denotes a b-bit input DAC with input-noise standard deviation s.

A.1 Primary Llama QA testbeds

Both primary testbeds use an equal mixture of NQ-Open, TriviaQA, and WebQuestions. Fine-tuning uses 2,000 optimizer steps, sequence length 256, micro-batch size one, digital aggregation over eight micro-batches, LoRA rank $r = 8$, and scaling factor $\alpha = 16$. The base learning-rate amplitude is 10^{-4} and decays to zero. Muon uses momentum 0.95, while AdamW uses $(\beta_1, \beta_2) = (0.9, 0.999)$ and weight decay 0.01. The corresponding Q/K/V/O projections and LoRA weights are mapped to analog arrays, while input reshaping is applied to Q/K/V. Calibration uses 64 pre-fine-tuning batches, $\lambda = 0.8$, and the eight channels most frequently attaining the largest absolute token activation. The calibration statistics and selected channels remain fixed during fine-tuning. The primary noisy-I/O setting uses 7-bit input DACs and input-noise standard deviation 0.02.

Evaluation uses greedy generation on a fixed 512-question QA set with standard answer normalization and best matching over gold-answer aliases. Validation NLL is computed on 100 held-out examples using response-token negative log-likelihood.

Llama-3-8B. For the original softmax-attention Llama-3-8B, the Q/K/V/O projections and corresponding LoRA weights in the first six decoder blocks are mapped to analog arrays. The primary experiments use $N = 100$. For the $N = 20$ stress test, AdamW uses a fivefold learning-rate multiplier relative to the $N = 100$ setting.

Llama-3.2-1B-Instruct. For Llama-3.2-1B-Instruct, Q/K/V/O projections and their corresponding LoRA weights in all 16 decoder blocks are mapped to analog arrays. MLPs, embeddings, normalization layers, attention softmax, and the language-model head remain digital. The analog device uses $N = 100$ with programming increment $\Delta w_{\min} = 0.02$.

A.2 Additional testbeds

Llama/OpenOrca. We fine-tune the softmax-attention Llama-3-8B on OpenOrca using an instruction–response format. The last eight decoder blocks' Q/K/V projections and corresponding LoRA weights are mapped to analog arrays. Input reshaping uses $\lambda = 0.8$ and eight calibrated outlier channels. Evaluation reports final response-token NLL. The evaluated analog settings are given in Table 6.

Recovered linear-attention Llama QA. We additionally evaluate a linear-attention Llama-3-8B checkpoint recovered through LoLCATs [41, 42] on the same three QA datasets. The first six decoder blocks' Q/K/V/O projections and corresponding LoRA weights are mapped to analog arrays, with reshaping applied to Q/K/V.

B Additional Results

B.1 Additional QA Results

Table 4 reports validation NLL for the primary QA experiments and shows the same qualitative trends as F1/EM: reshaping improves noisy-I/O performance, while discarding the programming residual substantially degrades performance. Table 5 extends the evaluation to $N = 20$, where both reshaping and update accumulation remain beneficial.

Table 4: **Validation NLL for the primary QA experiments.** Lower is better.

Model	I/O	Optimizer	Validation NLL		
			No reshaping + UA	Reshaping + UA	Reshaping w/o UA
Llama-3-8B	Perfect	Muon	1.8349	1.8250	–
Llama-3-8B	Perfect	AdamW	1.8728	1.8650	–
Llama-3-8B	7/0.02	Muon	2.3346	**1.9617**	4.5610
Llama-3-8B	7/0.02	AdamW	2.5149	**2.1374**	4.4824
Llama-3.2-1B	7/0.02	Muon	3.0193	**2.9235**	6.7572
Llama-3.2-1B	7/0.02	AdamW	3.1864	**3.0138**	6.3773

Table 5: **Llama-3-8B QA at $N = 20$.** F1 and EM are in %. Results are point estimates.

	Muon			AdamW		
Configuration	F1	EM	NLL	F1	EM	NLL
No reshaping + UA	18.45	12.70	3.1689	26.89	20.31	3.0256
Reshaping + UA	**39.96**	**31.84**	**2.3762**	**39.41**	**31.84**	**2.1780**
Reshaping w/o UA	31.08	21.29	4.4745	30.26	20.70	4.5044

B.2 Additional Instruction-Tuning Results

Table 6 extends the instruction-tuning evaluation to Llama-3-8B/OpenOrca. Reshaping lowers mean response-token NLL in all six noisy within-optimizer comparisons, consistent with the Qwen/OpenThoughts results in Table 2.

Table 6: **Llama-3-8B/OpenOrca response-token NLL.** Mean $\pm$ standard deviation over three seeds; lower is better. UA is enabled throughout.

N	I/O setting	Muon [NLL↓]		AdamW [NLL↓]	
		No reshaping	Reshaping	No reshaping	Reshaping
10^4	Perfect	0.6841$\pm$0.0024	–	0.7001$\pm$0.0008	–
100	Perfect	0.7164$\pm$0.0012	–	0.7373$\pm$0.0014	–
100	4/0.01	0.7355$\pm$0.0021	**0.7239**$\pm$0.0015	0.7564$\pm$0.0025	**0.7429**$\pm$0.0024
100	6/0.02	0.7302$\pm$0.0021	**0.7211**$\pm$0.0019	0.7488$\pm$0.0024	**0.7419**$\pm$0.0023
10^4	6/0.02	0.7048$\pm$0.0015	**0.6941**$\pm$0.0001	0.7151$\pm$0.0023	**0.7070**$\pm$0.0039

B.3 Recovered Linear-Attention QA Results

Tables 7 and 8 report the complete recovered-QA results. F1 and EM each improve in seven of eight noisy comparisons. The exceptions are AdamW EM at $N = 100$, 6/0.01, and AdamW F1 at $N = 20$, 6/0.01; both metrics improve together in six comparisons.

Table 7: **Recovered linear-attention Llama-3-8B QA.** Results at $N \in \{100, 10^4\}$ are mean $\pm$ standard deviation over three seeds.

N	I/O	Muon		AdamW	
		No reshaping + UA	Reshaping + UA	No reshaping + UA	Reshaping + UA
10^4	Perfect	51.58$\pm$1.4 / 44.33$\pm$1.4	–	51.84$\pm$0.7 / 44.53$\pm$1.1	–
100	Perfect	50.67$\pm$1.2 / 43.69$\pm$0.5	–	49.95$\pm$0.9 / 42.64$\pm$0.6	–
100	7/0.02	46.84$\pm$1.3 / 39.45$\pm$1.3	50.81$\pm$0.3 / 43.49$\pm$0.8	45.00$\pm$0.4 / 38.35$\pm$0.9	49.06$\pm$0.2 / 41.73$\pm$0.5
100	6/0.01	49.76$\pm$1.6 / 42.06$\pm$1.4	50.72$\pm$0.2 / 43.29$\pm$0.2	48.42$\pm$0.5 / 41.60$\pm$0.6	48.85$\pm$0.8 / 41.15$\pm$0.2

Table 8: **Recovered linear-attention Llama-3-8B QA at $N = 20$.** Results are point estimates.

N	I/O	Muon		AdamW	
		No reshaping + UA	Reshaping + UA	No reshaping + UA	Reshaping + UA
20	Perfect	49.01 / 42.38	–	50.00 / 43.16	–
20	7/0.02	40.42 / 33.40	48.08 / 41.41	45.85 / 38.09	49.64 / 42.19
20	6/0.01	47.54 / 41.21	48.77 / 41.80	48.20 / 40.62	47.95 / 41.02

C Input-Reshaping Ablations and Diagnostics

C.1 Reshaping ablations

Table 9 compares base-path rescaling alone with full reshaping. Base-path rescaling provides most of the observed improvement; adding outlier-channel splitting further improves both F1 and EM.

Table 9: **Input-reshaping component ablation.** Llama-3-8B QA with Muon under $N = 100$ and I/O setting 7/0.02. Results are point estimates of F1 (EM), in %. BR denotes function-preserving base-path rescaling and OS denotes outlier-channel splitting.

BR Off, OS Off	BR On, OS Off	BR On, OS On
24.49 (18.36)	42.01 (33.79)	**43.55 (35.35)**

C.2 Calibration stability

Figure 2 compares the fixed pre-fine-tuning top-8 outlier set with rankings recomputed over 64-step training windows. Channels are ranked by how often they attain the largest absolute token activation. Most selected channels remain shared with the calibration set over the plotted interval.

C.3 Quantization-noise compression

Figure 3 reports analytical ratios for the additive DAC quantization-noise component in Eq. (2), computed from recorded inputs and weights. Each cell averages per-token ratios within a 200-step window. For outlier-channel splitting, the calibrated channels $\mathcal{L}$ use the protected path. At fixed DAC precision, the per-channel quantization-noise variance ratio on the remaining analog inputs is

$$\left(\frac{\|\mathbf{x}_{\mathcal{L}^c}\|_\infty}{\|\mathbf{x}\|_\infty} \right)^2 . \tag{6}$$

For base-path rescaling, let $\overline{\mathbf{x}} = \mathbf{D}^{-1}\mathbf{x}$ and $\overline{\mathbf{W}}^{\text{base}} = \mathbf{W}^{\text{base}}\mathbf{D}$. Under independent, equal-variance additive quantization errors, the modeled output-noise energy ratio is

$$C_y(\mathbf{x}) = \left(\frac{\|\overline{\mathbf{x}}\|_\infty}{\|\mathbf{x}\|_\infty} \right)^2 \frac{\|\overline{\mathbf{W}}^{\text{base}}\|_F^2}{\|\mathbf{W}^{\text{base}}\|_F^2}, \tag{7}$$

where $\|\cdot\|_F$ is the Frobenius norm. The two factors account for changes in input range and weight norm. The plotted ratios are approximately 0.61–0.82 for splitting and 0.22–0.33 for base-path rescaling, remaining below one throughout the displayed windows.

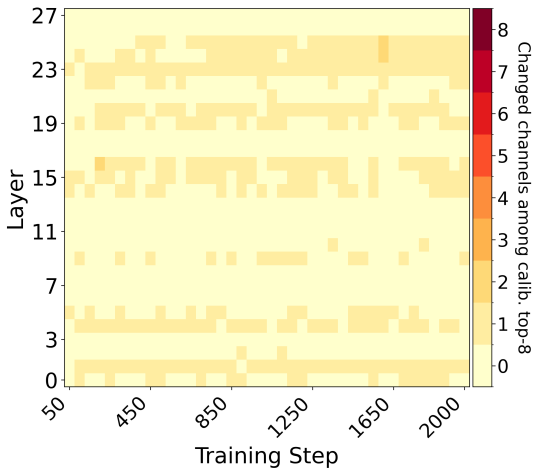


Figure 2: **Calibration stability on Qwen/OpenThoughts.** Each cell counts channels in a 64-step window's top-8 set that are absent from the calibration set; zero indicates full overlap. Rows are monitored layers and columns are training windows.

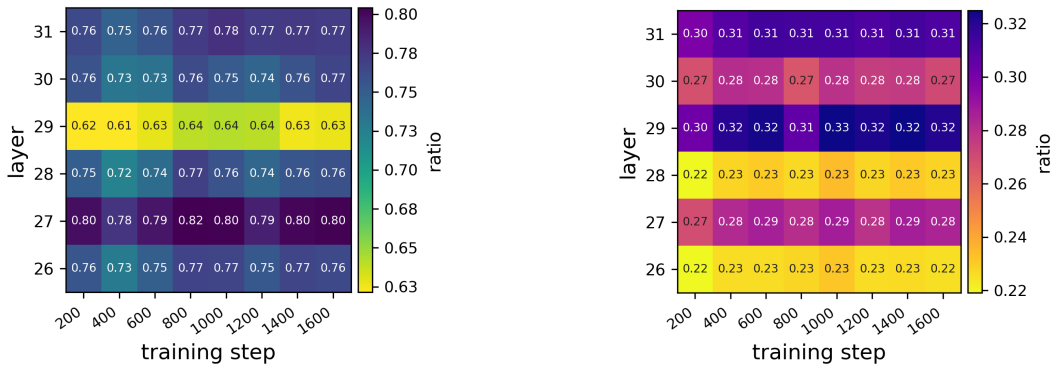


Figure 3: **Input-reshaping quantization-noise diagnostics.** Left: remaining analog channels' input-noise variance ratio (Eq. (6)). Right: base-path output-noise energy ratio, including weight rescaling (Eq. (7)). Cells average per-token ratios within 200-step windows; values below one indicate compression.